박성은
BADA

Cloud Infra · 운영 · 비용 설계

문제를 다시 정의하고, 만들지 않을 것부터 정합니다.

기간·인원 같은 조건을 먼저 적고 그 안에서 범위를 자릅니다. 기각한 선택과 그 근거를 함께 남겼습니다.

Frontend · UI/UX | Infra | Data · AI | PM · 기획

Web sungeun-portfolio.vercel.app
Email pppp000310@gmail.com
GitHub github.com/bada0310
Blog velog.io/@sungeun0310
Service www.dero.life
Education 한국외대 산업경영공학 · SSAFY 15기
Language OPIc IH

00 한장요약

프로젝트 10 · 수상 3
교육 1,735시간

대표 성과

38MB → 378KB
3D 에셋 용량
원본 대비 약 1/100 · 스마트 토폴로지 경량화 (DERO)

수 초 → 0
배포 중 다운타임
blue-green 전환 도입 전후 (DERO)

-20%
월 운영 비용
초기 산정 27만 원 대비 · NAT Gateway 재산정 (AWS)

3~4h → 2h
스터디 회차 준비시간
출제·채점·집계 자동화 (퀴즈 쇼)

직무 축별 대표 프로젝트

FRONTEND · UI/UX
DERO 3D 배치 시뮬레이터 · 렌더 성능 예산 | **멍경찰과 냥도둑** 입력 채널 설계 | **실시간 퀴즈 쇼** 습관을 안 바꾸는 설계

INFRA
AWS 예약 서비스 멀티 리전 · 비용 구조 분해 | **DERO** EC2 1대 무중단 배포 | **Lumière** 터널 고정화

DATA · AI
Bimp 인식 한계를 전제로 한 설계 | **영화 추천(NCF)** 과적합 판정 | **인천 카페** 설명 가능성 우선

기술 — 주력 / 경험

`Python` `AWS` `Docker` `Nginx` `Jenkins` `Spring Boot` `Next.js` `Three.js` `PyTorch` `YOLOv5` `Elasticsearch` `Redis` `PostgreSQL` `Django` `Prometheus` `Grafana` `Unity` `Jira`

검은 칩 = **주력**(막혔을 때 스스로 원인을 찾아 끝까지 간 기술) · 흰 칩 = 팀 프로젝트에서 써본 기술

> 전체 내용과 다이어그램은 웹에서 볼 수 있습니다 — `sungeun-portfolio.vercel.app`

02 AWS 식당 예약 서비스

2024.11 – 12
4인 (BE 2 · FE 2) · 팀장 · 60%

배포로 끝내지 않고 장애 추적·확장성·비용까지 관리 가능한 구조로. **“안정성을 얼마나 살 것인가”**를 기술 선택이 아니라 기준 설정 문제로 놓았습니다. **Cloud DX Academy 프로젝트 경진대회 최우수상**

핵심 성과

월 운영비 약 20% 절감

산정표를 항목별로 뜯어 **NAT Gateway가 단일 최대 항목(월 약 $160)**임을 특정 — ALB $48, EC2 $47을 합친 것보다 큼. “트래픽이 없어도 나가는 고정비”부터 재산정 → **약 20% 절감**

한 리전이 죽어도 서비스 지속

3-Tier **Active-Standby**. Active-Active는 고정비 두 배라 명시적 기각

IP 대신 이름으로 찾게 전환

오토스케일링으로 IP가 바뀔 때마다 끊기던 라우팅을 **ECS Service Connect**로 전환 — 수동 갱신 **0회**

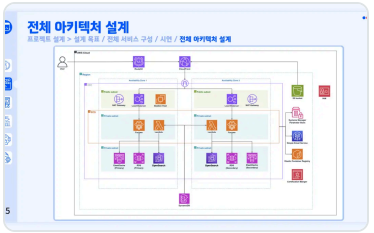
흩어진 로그를 한곳에서

Fluent Bit 사이드카 → OpenSearch → Kibana + Slack으로 “어디서 터졌는지 모르는” 상태 해소

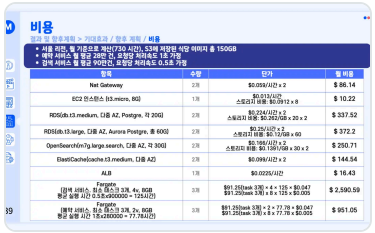
나의 역할	팀장 / 클라우드 인프라 아키텍트 / 백엔드
먼저 한 일	허용 다운타임과 월 예산 상한을 팀 합의로 확정
BACKEND	Java · Docker · REST API
DATA	Aurora (예약 · 정합성) · DynamoDB (조회 · 세션)
DEVOPS	ECS · Fargate · ALB · CloudFront · Route 53 · VPC · NAT Gateway
OBSERVABILITY	CloudWatch · Fluent Bit · OpenSearch · Kibana · Slack



단일 리전 — 열려야 하는 것(CloudFront · ALB)과 열리면 안 되는 것(DB · 앱)을 서브넷으로 물리 분리. 감수한 것은 NAT 게이트웨이 비용. 컨테이너는 재시작되므로 로그를 stdout으로 밖에 뻐다.



3-Tier × 멀티 리전. 외부에 열리는 건 ALB까지



예상 사용량에서 리소스별 적정 용량 역산



Fluent Bit 사이드카 → OpenSearch

기준이 없으면 모든 확장이 정당해 보입니다. 안정성은 사고 싶은 만큼 살 수 있어서 상한선이 없으면 끝없이 커집니다. 그래서 구성을 정하기 전에 **허용 다운타임과 월 예산 상한**을 숫자로 먼저 합의했고, 이후 모든 선택을 그 기준으로 판정했습니다.

02 AWS · 기술 의사결정

구성 대안 비교

대안	얻는 것	치르는 것	판정
A. 단일 리전 + 백업	가장 저렴	리전 장애 시 서비스 전체 정지	기각
B. 멀티 리전 Active-Active	전환 시간 거의 0	두 리전 상시 구동 → **고정비 두 배**	기각
C. 멀티 리전 3-Tier (Active-Standby)	리전 장애 대응 범위 확보	전환 시간 · 복제 지연	**채택**

Q. 왜 ECS + Fargate인가 — EC2 직접 관리와 EKS를 두고

선택 기준은 단 하나, **"운영할 사람이 없다"** 였습니다. EC2 직접 배포는 OS 패치·스케일·헬스체크가 전부 우리 몫이고, EKS는 컨트롤 플레인 고정비에 학습·운영 비용이 프로젝트 기간을 넘겼습니다.

성과 서버 인스턴스를 관리 대상에서 제거 — 인프라 전담 없이도 배포가 돌아감

감수 AWS 종속이 깊어집니다. 기간과 인력이 제약인 프로젝트에서 **이식성은 지금 사지 않아도 되는 보험**이라고 판단했습니다.

Q. Aurora와 DynamoDB를 왜 같이 쓰나

예약은 중복이 생기면 안 되는 **정합성** 문제이고, **조회·세션**은 요청이 몰리는 **속도** 문제입니다. 하나로 통일하면 둘 중 하나는 잘못된 도구를 씁니다.

기준 DERO에서는 정반대로 PostgreSQL 하나로 합쳤습니다. 거기선 두 성격의 데이터를 **같은 조회에서 같이** 봐야 했고, 여기선 **서로 다른 요청 경로**에 있었습니다. 기준은 '데이터의 종류'가 아니라 **'같이 조회되는가'** 였습니다.

트러블슈팅 — 뒷단으로 가는 요청이 간헐적으로 실패했다

증상	게이트웨이에서 뒷단으로 가는 요청이 간헐적으로 실패했습니다.
처음 가설	백엔드가 죽었다고 봤습니다. **틀렸습니다** — 태스크는 살아 있었고 주소만 바뀌어 있었습니다.
실제 원인	오토스케일링으로 컨테이너가 재생성될 때마다 **IP가 변경**됐습니다.
기각한 안	IP 고정은 **오토스케일링을 포기하는 것**과 같고, 서비스마다 내부 ALB는 또 다른 고정비였습니다.
해결	**ECS Service Connect** — IP를 고정하는 방향이 아니라 **이름으로 찾는 방향**으로 바꿨습니다.
배운 것	"주소가 변한다"를 막을 문제로 보지 않고 **전제로 받아들이자** 해법이 바뀌었습니다.

회고

잘한 점

구성을 정하기 전에 **비용과 안정성의 기준을 팀 합의로 먼저 정한 것.** 이후 모든 선택이 그 기준으로 판정됐습니다.

아쉬운 점

관측을 **설계 단계가 아니라 문제를 겪은 뒤에** 붙였습니다. 실제 부하도 재보지 않고 예상 사용량만으로 산정했습니다.

배운 점

관측은 기능이 아니라 **전제**입니다. 분산 환경에서 "어디서 터졌는지 모른다"는 상태 자체가 장애입니다.

다음에는

컨테이너를 2개 이상 띄우기로 결정한 **그 시점에 로그 경로를 같은 그림에** 그립니다.

01 DERO — 데스크테리어 3D 시뮬레이터

2026.07 – 08 (6주)
5인 · 팀원 · 기여도 40%

제품을 고르는 화면과 내 책상에 놓아보는 화면을 하나로 합친 서비스. EC2 **한 대**에서 무중단 배포까지 끌고 갔습니다.
SSAFY 공통 프로젝트 경진대회 우수상 · 서비스 운영 중(dero.life)

핵심 성과

배포 다운타임 수 초 → 0
호스트 nginx에 포트가 하드코딩돼 제자리 교체되던 구조를 **blue-green**으로 재설계

GLB 38MB → 378KB
POC 실측으로 소품당 **20K~50K 삼각형** 예산 확정

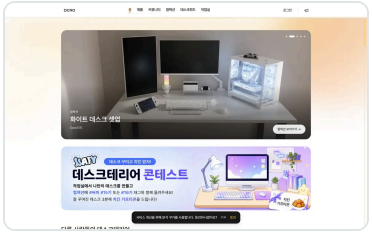
핵심 기능 검증 7일 → 3일
GLB 변환·렌더링을 전체 개발 전 POC로 분리해 먼저 검증

배포 계정에 최소 권한만
컨테이너 `127.0.0.1` 바인딩으로 평문 노출 제거, jenkins sudo는 **두 명령만** 허용

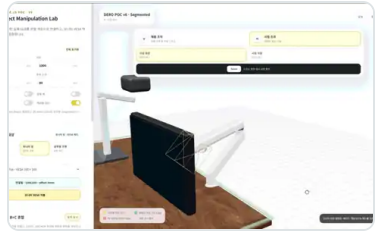
나의 역할	**기획(PM) · 프로젝트 관리** · 백엔드 · 인프라 · CI/CD
제약	EC2 1대 · 운영 인력 1명 · 6주
FRONTEND	Next.js (SSR) · Three.js · glTF/GLB · KTX2
BACKEND	Spring Boot · PostgreSQL (JSONB) · Redis · Elasticsearch · Flyway
DEVOPS	EC2 · Docker Compose · Jenkins · Nginx (blue-green)
OBSERVABILITY	Prometheus · Grafana · Filebeat · Kibana

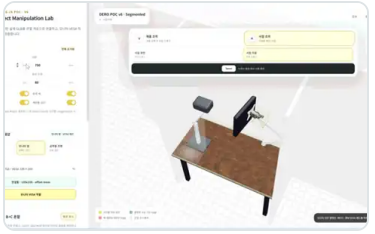


무중단 배포 파이프라인 — MR 하나가 머지되면 사람이 서버에 들어가지 않는다. 포트 두 벌(blue 3000/8081 · green 3010/8091)을 번갈아 쓰고, 헬스체크를 통과한 뒤에만 nginx upstream을 스위치한다. 통과하지 못하면 green을 폐기하고 blue를 유지 — 트래픽이 전환되지 않으므로 요청 단절이 없다.



제품 탐색 → 상세 → 3D 배치를 하나의 시나리오로 고정


VESA 마운트홀이 맞을 때만 스냅 — 호환성은 규격 문제


실측 치수로 책상을 맞춘 뒤 배치

01 DERO · 트러블슈팅 · 회고

배포할 때마다 서비스가 몇 초씩 끊겼다

증상	배포할 때마다 서비스가 몇 초씩 끊겼습니다.
처음 가설	Jenkins 배포 스크립트가 느려서라고 봤습니다. **틀렸습니다** — 스크립트를 줄여도 끊김은 그대로였습니다.
실제 원인	nginx가 호스트에 있고 `127.0.0.1:3000·8081` 이 하드코딩돼 있어, `docker compose -p dero up -d` 가 **같은 컨테이너를 제자리 교체**하고 있었습니다. 구조상 끊길 수밖에 없었습니다.
어떻게 찾았나	배선을 위에서 아래로 되짚었습니다 — nginx conf의 `proxy_pass` → compose 포트 바인딩 → Jenkinsfile의 `-p` 프로젝트명.
해결	포트 변수화 + compose의 `name: dero` 삭제 + upstream conf 두 벌 + `deploy-switch.sh`. health 폴링을 통과해야만 트래픽이 넘어가고, 실패하면 blue가 그대로 유지됩니다.
배운 것	무중단 배포는 '배포 도구' 문제가 아니라 **'포트와 라우팅' 문제**였습니다. 그리고 구조를 바꾸면 **관측도 따라 흔들립니다.**

알려진 부작용까지 기록했습니다

Prometheus가 `backend:8080`을 노려, blue·green이 겹치는 수십 초 동안 지표가 두 버전 사이를 라운드로빈합니다. 색별 별칭 + 타깃 2개로 갈 수 있지만 평상시 한쪽이 DOWN으로 뜹니다. **겹침 창이 짧아 문서에 남기는 쪽**을 택했습니다.

팀 리딩

각자 무엇을 하는지 주 단위로만 알 수 있어 겹치는 작업과 대기가 생겼습니다. 사람이 아니라 구조 문제였습니다 — 업무 단위가 '기능'이라 병렬화가 안 됐습니다. 주간/데일리 스크럼 · Jira 이슈 컨벤션 + 스토리 포인트 · API 명세 사전 정리를 제안했습니다. **결과: 요구 기능이 3개 → 6개로 늘었을 때 기존 스토리에 추가만으로 대응, 일정 지연 없음.**

회고

잘한 점

코드보다 **ADR을 먼저 썼습니다.** blue-green을 켜기 전 마이그레이션 정책부터 정한 덕에 첫 스키마 변경에서 파이프라인이 멈추지 않았습니다.

아쉬운 점

메모리 실측이 뒤였습니다. 6개 프로세스가 이미 올라간 EC2에 한 벌을 더 띄우는 결정을 `free -m`을 보기 전에 내렸습니다.

배운 점

발표 리허설에서 "경량화를 어떻게 했는지 설명이 없다"는 지적을 받았습니다. **판단은 했는데 판단의 기준을 남기지 않은 것**이 문제였습니다.

다음에는

자원 실측 → 정책(ADR) → 코드 순서로 갑니다. 결과 숫자를 쓸 때 기준을 같은 줄에 적습니다.

03 Bimp — 냉장고 식자재 인식 · 레시피 추천

2024.03 – 06 (3개월)
4인 (PM·앱 1 · 웹 1 · AI 2) · 40%

직접 입력의 불편을 이미지 인식으로 줄인 졸업 프로젝트. 문제를 '추천'이 아니라 **'등록'**으로 재정의한 데서 모든 설계가 따라 나왔습니다. **대한산업공학회 캡스톤디자인 경진대회 장려상**

핵심 성과

병목은 추천이 아니라 등록이었다

팀이 '추천 정확도'를 보고 있을 때 병목이 **그 이전 단계인 '등록'**에 있다고 짚고 스코프를 옮겼습니다

사진과 영수증을 한 결과로

객체 인식(YOLOv5)과 문자 인식(OCR)을 한 모델에 합치지 않고 **출력 형식만 공통 스키마로 통일** — 뒷단이 입력 방식을 몰라도 됨

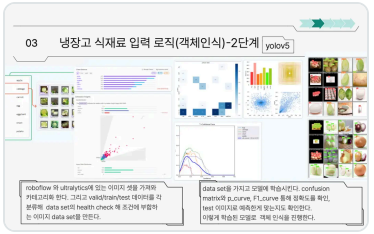
틀릴 것을 전제하고 8종만 자동화

깊이 카메라·RFID를 각각 비용·현실성으로 기각한 뒤, 인식 범위를 **8종으로 좁혀** 확실한 것만 자동 처리하고 **보정 단계**를 넣었습니다

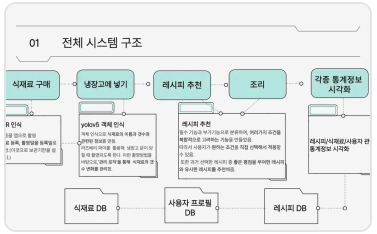
정확성 22/0/0 · 만족 14/5/3

사용성은 보통이 과반 — 숫자를 보고 나서야 **약점이 추천이 아니라 화면**임이 드러났습니다

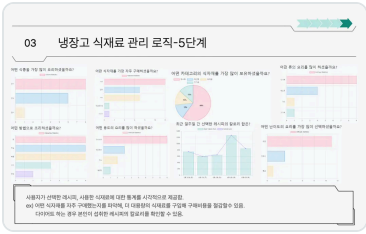
나의 역할	YOLO 인식 모델 · 추천 알고리즘 · 재고 로직
제약	학습 GPU 없음 · 카메라 1대 · 3개월
AI · VISION	YOLOv5 · PyTorch · Google Colab
BACKEND	Django · SQLite3 · REST API
DEVICE	Raspberry Pi · 카메라 모듈 · 마그네틱 센서



냉장고 내부에서 식재료 후보 검출 — 반드시 틀리므로 보정 전제



두 입력 경로를 하나의 공통 스키마로 합류



추가·삭제·수정을 분리한 보정 화면

트러블슈팅 — 겹치거나 잘린 식재료를 인식하지 못했다

증상	겹쳐 있거나 반 개로 잘린 식재료의 개수가 틀리게 잡혔습니다. 재고가 틀리면 추천 전체가 무의미해집니다.
처음 가설	학습 데이터를 늘리면 해결된다고 봤습니다. **틀렸습니다** — 데이터 양의 문제가 아니었습니다.
실제 원인	단안 카메라에는 **깊이 정보가 없습니다.** 가려지면 뒤에 무엇이 몇 개 있는지 원리상 알 수 없습니다.
기각한 안	① **3D 깊이 카메라** — 깊이는 얻지만 전 칸 설치 시 예산 초과 ② **RFID 태그** — 장 볼 때마다 부착해야 해서 원래 문제(등록이 번거롭다)를 더 키움
해결	정확도를 올리는 대신 **틀릴 것을 전제로 흐름을 바꿨습니다.** 인식 8종으로 축소 + 확정 전 보정 단계 + 검증 범위를 한 칸으로 한정.
배운 것	센서가 관측할 수 없는 것은 모델을 키워도 알 수 없습니다. **'정확하게'와 '틀려도 되돌릴 수 있게'는 다른 목표**이고, 어느 쪽인지 먼저 정했어야 했습니다.

06 학력 · 경력 · 수상

학력 · 교육

2019.03 – 2025.07
한국외국어대학교 산업경영공학과
학사 졸업 · 공정과 서비스 흐름의 낭비를 찾아 재설계하는 방법론

2026.01 – 2026.07
삼성 청년 SW·AI 아카데미 (SSAFY) 15기
925시간 · 대전

2024.07 – 2024.12
교보DTS Cloud Architecture DX Academy
810시간 · Linux · 네트워크 · Docker/K8s · CI/CD · AWS

수상 · 어학

2026
SSAFY 공통 프로젝트 경진대회 우수상
DERO · 5인팀

2024.12
Cloud DX Academy 프로젝트 경진대회 최우수상
교보DTS · 4인팀 팀장

2024.10
제20회 산업공학프로젝트 경진대회 장려상
대한산업공학회 · Bimp

2026.02
OPIc IH

경력

2025.08 – 2026.01
타임교육
교육자료 디지털화 · 전산화, 자료 분류 및 누락 · 형식 검수

2022.03 – 2025.08
바나프레소 · 카페 아르바이트
주중·주말 교대 3년 반. 피크 시간대 주문 처리와 재고·위생 점검

2022.08 – 2023.01
법무법인 화우
판례 · 법률문서 디지털화, 쟁점별 라벨링 및 DB 구축 지원

활동

2023.07 – 2025.07
ChADa · AI 연구 스터디
한국외국어대학교 · 논문 리뷰, 모델 구현 및 발표

2026.03 – 04
CS 스터디 운영 개선
노션 글 → 문제화 → 실시간 퀴즈 사이트. 준비시간 3~4h → 2h

> **일하는 방식.** 역할은 매번 달랐지만 진입 방식은 같았습니다 — **AS-IS를 계측하고 → TO-BE를 정의하고 → 가장 작은 실험으로 확인하고 → 절차로 굳힙니다.**

연락처

EMAIL pppp000310@gmail.com
WEB sungeun-portfolio.vercel.app
GITHUB github.com/bada0310
BLOG velog.io/@sungeun0310

포스텔의 법칙 · 견고함의 법칙
Be conservative in what you do, be liberal in what you accept from others.

상세 산출물(ERD 설계서, 아키텍처 문서, 배포 트러블슈팅 기록, 각 프로젝트 보고서)은 요청 주시면 보내드립니다.
DERO 소스는 SSAFY 사내 GitLab에 있어 외부 열람이 제한됩니다 — 공개 미러 준비 중입니다.